

CodeRay 보안약점 분석 상세 보고서

187_LLM 번역기 - 1 차 소프트웨어 개발보안 가이드 49개 항목

기관명	삼성물산건설
프로젝트 그룹 명	기본 그룹
프로젝트 이름	187_LLM 번역기
분석회차 (분석일)	1차 (2026-06-09)
보고서 출력일자	2026-06-09
담당부서 / 담당자	IT 보안/인프라지원센터 / -
분석 엔진 버전	v6.0.3.17

< 보고서 요약 >

보안약점 점검의 필요성

SW 개발 보안은 SW 개발 과정에서 개발자 실수, 논리적 오류 등으로 인해 SW에 내포될 수 있는 보안 약점 원인, 즉 보안 약점을 최소화 하는 한편, 사이버 보안 위협에 대응할 수 있는 안전한 SW를 개발하기 위한 일련의 보안 활동을 의미합니다. 광의적 의미로는 SW 개발 생명 주기(SDLC, Software Development Lifecycle)의 각 단계 별로 요구되는 보안 활동을 모두 포함하며, 협의적 의미로는 SW 개발 과정 중 소스코드 구현 단계에서 보안 약점을 배제하기 위한 '시큐어코딩(SecureCoding)'을 의미합니다. 최근의 사이버 공격은 침입차단 시스템 등 보안 장비를 우회하거나, 보안 패치가 발표되기 이전의 보안 약점을 악용하는 제로데이 공격, 웹사이트 해킹 등이 많은 부분을 차지하고 있으며 사이버 공격의 약 75%가 SW자체의 보안 약점을 악용하는 것으로 웹사이트 공격이 대표적 예라고 할 수 있습니다.

사이버 공격을 선제적으로 예방 및 대응하기 위해서는 제품 출시 이전 단계인 SW 개발단계에서 보안 약점을 제거하는 것이 가장 효과 적인데, 이는 미국 국립표준 기술연구 소(NIST)의 연구 결과에 따르면 설계 과정에서 발생한 결함이 개발완료 후에 발견, 조치 되는 경우에는 설계 단계에서 결함을 수정 하는 비용대비 30배의 비용이 발생 한다고 합니다.또한 통합 과정에서 발생한 결함의경우, 20배의 수정비용이 발생하는등 개발 완료 이전에 보안 약점을 진단, 제거 하는 활동인 'SW개발 보안' 이 무엇보다 중요함을 나타내고 있습니다.

보안약점 점검 방식과 구성

CODE-RAY는 정적분석 시큐어코딩 솔루션으로, SW를 실행하지 않고 소스 코드 수준으로 보안 약점을 분석합니다.정적분석 방법은 SW 개발 초기에 보안 약점 발견으로 빠르고 적은 비용으로 소스 코드 수정이 가능한 장점이 있지만,컴포넌트간 발생할 수 있는 통합된 보안 약점 발견이 제한적일 수 있고 설계, 구조 관점의 보안 약점은 발견할 수 없습니다.

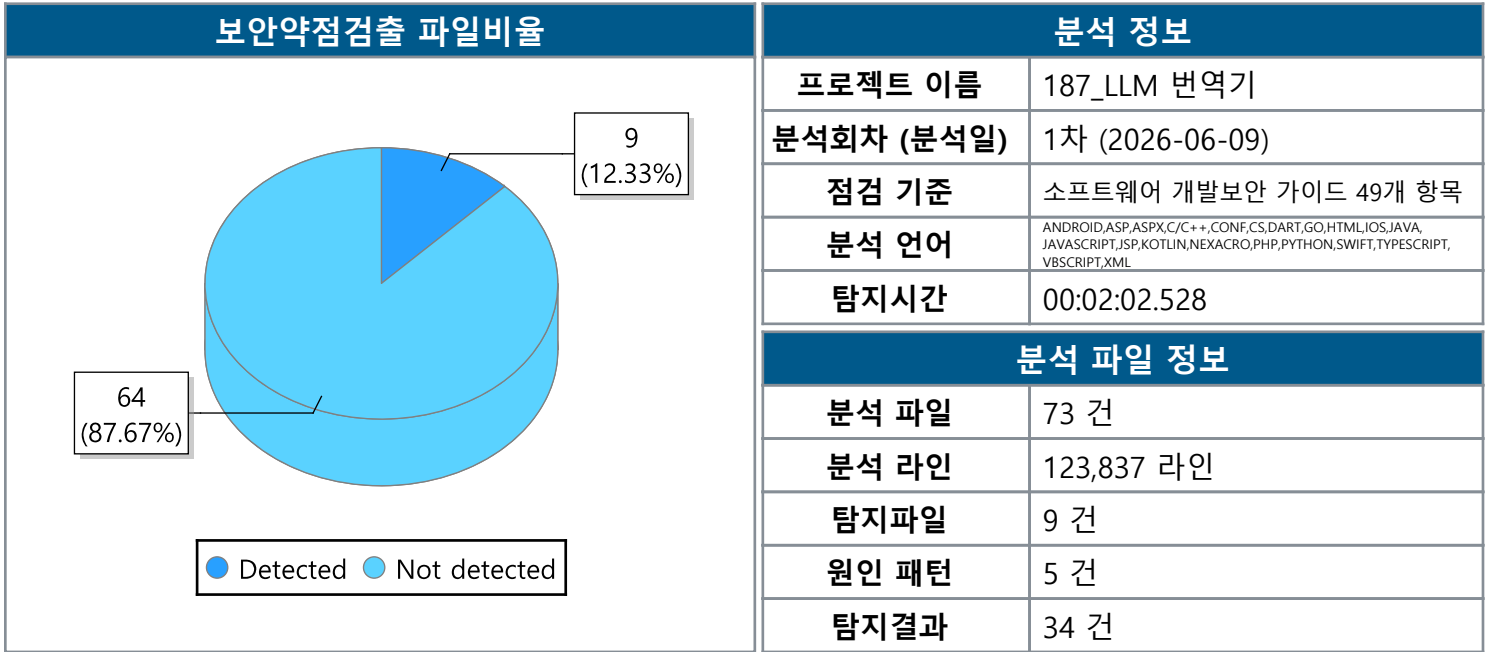
CODE-RAY는 위와 같은 단점을 해결하기 위해 소스코드의 트리구조화 및 변수추적, 흐름 추적등의 기법으로 보안약점 발견의 확률이 높고, 서버환경 설정, 프레임워크 설정등의 보안약점 및 코드품질도 탐지해 SW 구조관점의 보안약점도 탐지 가능합니다.

분석유형 으로 '행정안전부 SW 보안약점 49개 기준','OWASP TOP10','CWE/SANSTOP 25','JAVA Code Convention','국정원 홈페이지 8대 보안약점', ' 전자금융감독규정 8대 보안약점'등의 기준에 의하여 점검 가능합니다.

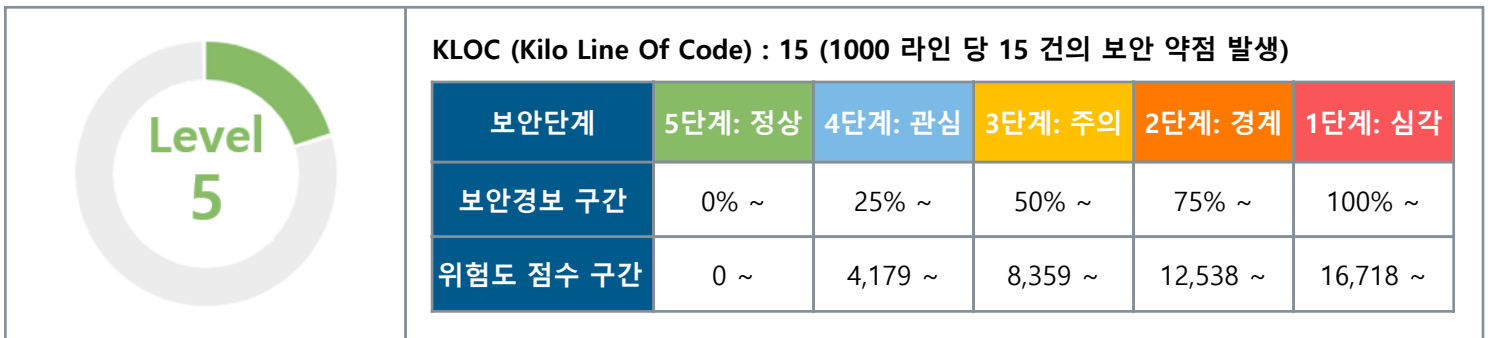
위험도 분류 기준

심각	보안 설정 미흡으로인하여 조직의정보보호관리 및정보자산에 직접적인피해를 입을 수 있으며,중요한 자산 및 정보가노출될 수 있는 매우심각한 단계의 보안약점
높음	보안 설정 미흡으로 인하 여 조직의 정보보호관리 및 정보자산에 직접적인 피해를 입을 수 있으며, 중요한 자산 및 정보가 노출될 수 있는 단계의 보안 약점
중간	보안 설정 미흡으로인하여 조직의정보보호관리 및정보자산에 간접적,부분적인 피해를 입을수 있으며, 공격자에게정보가 제공될 수 있는단계의 보안 약점
낮음	조직의 정보보호관리 및 정보자산에 직접/간접적인 피해를 예상하기 어려우나, 소프트웨어 품질 면에서 취약할 수 있는 단계의 보안 약점

1. 프로젝트 분석결과 요약



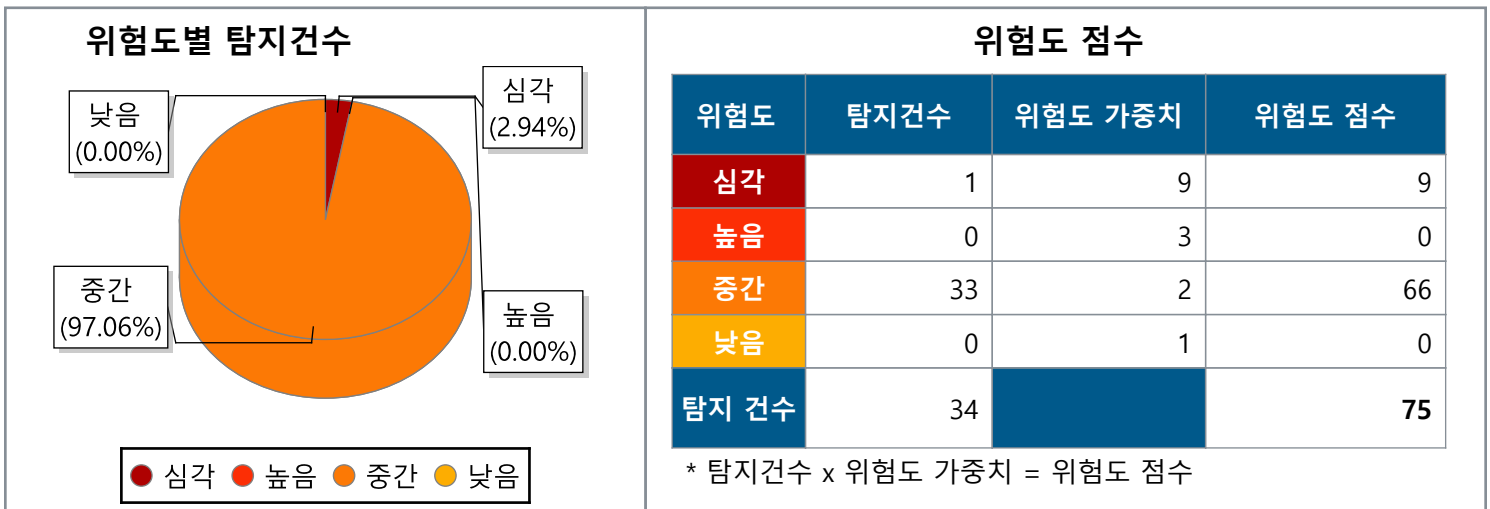
1.1 프로젝트 분석 보안경보 현황



* 보안경보 구간 : 소스코드의 총 라인수를 KLOC 15 기준으로 환산하여 5단계의 보안경보 구간 도출

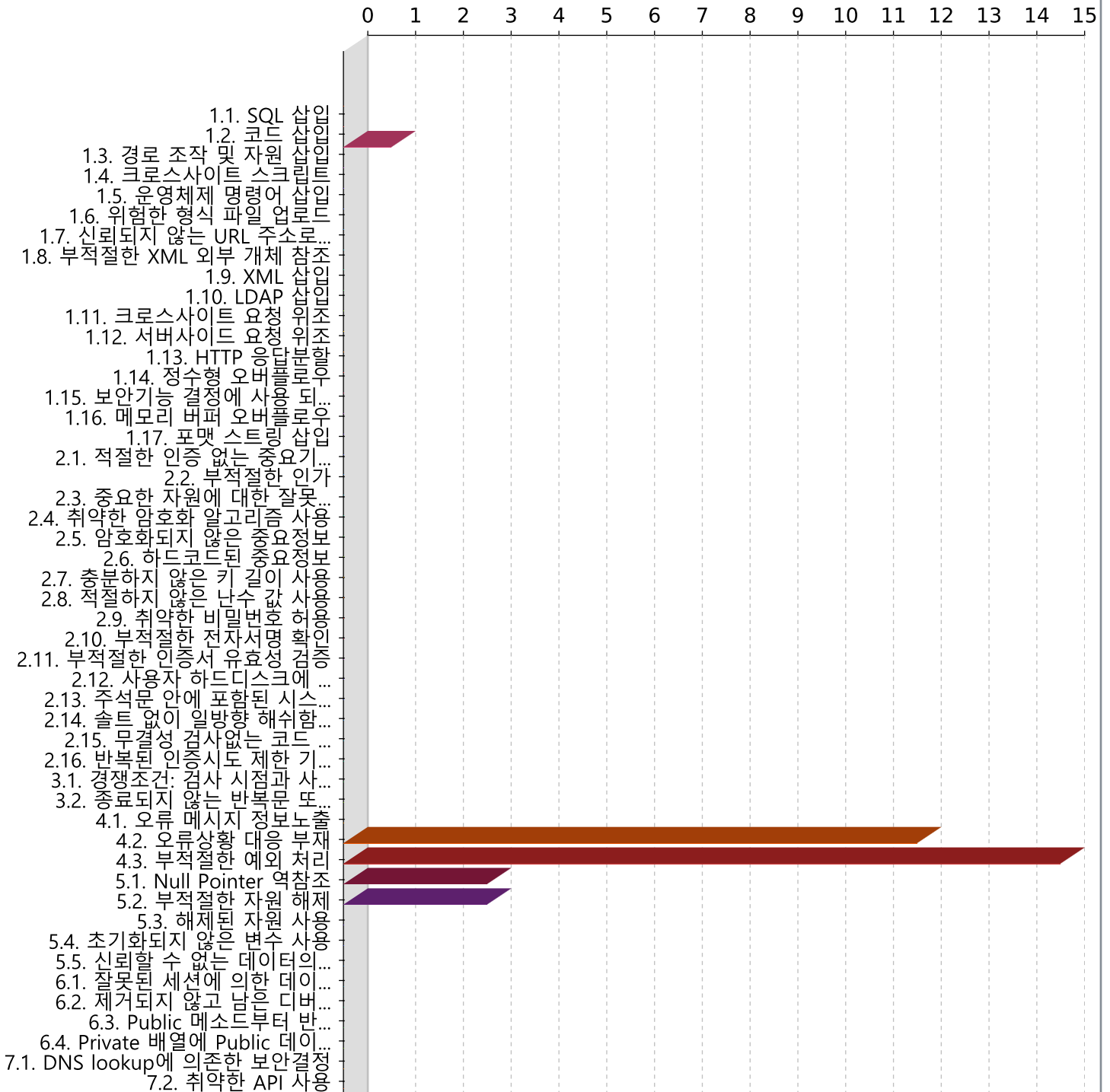
* 보안경보 레벨 : 위험도 점수의 총 합이 속하는 보안경보 구간 및 위험도 점수의 구간을 확인하여 보안경보 레벨 도출

1.2 프로젝트 분석 위험도 현황



1.3 프로젝트 점검 항목별 결과 요약

점검 항목 별 보안약점 탐지 건수



번호	점검 유형	원인패턴 건수	탐지건수
1	1.1. SQL 삽입	0	0
2	1.2. 코드 삽입	0	0
3	1.3. 경로 조작 및 자원 삽입	1	1
4	1.4. 크로스사이트 스크립트	0	0
5	1.5. 운영체제 명령어 삽입	0	0
6	1.6. 위험한 형식 파일 업로드	0	0
7	1.7. 신뢰되지 않는 URL 주소로 자동접속 연결	0	0
8	1.8. 부적절한 XML 외부 개체 참조	0	0
9	1.9. XML 삽입	0	0
10	1.10. LDAP 삽입	0	0
11	1.11. 크로스사이트 요청 위조	0	0
12	1.12. 서버사이드 요청 위조	0	0
13	1.13. HTTP 응답분할	0	0
14	1.14. 정수형 오버플로우	0	0
15	1.15. 보안기능 결정에 사용 되는 부적절한 입력값	0	0
16	1.16. 메모리 버퍼 오버플로우	0	0
17	1.17. 포맷 스트링 삽입	0	0
18	2.1. 적절한 인증 없는 중요기능 허용	0	0
19	2.2. 부적절한 인가	0	0
20	2.3. 중요한 자원에 대한 잘못된 권한 설정	0	0
21	2.4. 취약한 암호화 알고리즘 사용	0	0
22	2.5. 암호화되지 않은 중요정보	0	0
23	2.6. 하드코드된 중요정보	0	0
24	2.7. 충분하지 않은 키 길이 사용	0	0

번호	점검 유형	원인패턴 건수	탐지건수
25	2.8. 적절하지 않은 난수 값 사용	0	0
26	2.9. 취약한 비밀번호 허용	0	0
27	2.10. 부적절한 전자서명 확인	0	0
28	2.11. 부적절한 인증서 유효성 검증	0	0
29	2.12. 사용자 하드디스크에 저장되는 쿠키를 통한 정보 노출	0	0
30	2.13. 주석문 안에 포함된 시스템 주요정보	0	0
31	2.14. 솔트 없이 일방향 해쉬함수 사용	0	0
32	2.15. 무결성 검사없는 코드 다운로드	0	0
33	2.16. 반복된 인증시도 제한 기능 부재	0	0
34	3.1. 경쟁조건: 검사 시점과 사용 시점(TOCTOU)	0	0
35	3.2. 종료되지 않는 반복문 또는 재귀 함수	0	0
36	4.1. 오류 메시지 정보노출	0	0
37	4.2. 오류상황 대응 부재	1	12
38	4.3. 부적절한 예외 처리	1	15
39	5.1. Null Pointer 역참조	1	3
40	5.2. 부적절한 자원 해제	1	3
41	5.3. 해제된 자원 사용	0	0
42	5.4. 초기화되지 않은 변수 사용	0	0
43	5.5. 신뢰할 수 없는 데이터의 역직렬화	0	0
44	6.1. 잘못된 세션에 의한 데이터 정보 노출	0	0
45	6.2. 제거되지 않고 남은 디버그 코드	0	0
46	6.3. Public 메소드부터 반환된 Private 배열	0	0
47	6.4. Private 배열에 Public 데이터 할당	0	0
48	7.1. DNS lookup에 의존한 보안결정	0	0

번호	점검 유형	원인패턴 건수	탐지건수
49	7.2. 취약한 API 사용	0	0

2. 프로젝트 점검 항목별 결과

2.1 [1.3. 경로 조작 및 자원 삽입]

보안약점ID	위험도	보안약점	원인패턴 건수	탐지건수
CWE-22	심각	제한된 디렉토리에 대한 경로 이름의 부적절한 제한 ('Path Traversal')	1	1

2.2 [4.2. 오류상황 대응 부재]

보안약점ID	위험도	보안약점	원인패턴 건수	탐지건수
CWE-390	중간	조치없이 오류 조건 감지	1	12

2.3 [4.3. 부적절한 예외 처리]

보안약점ID	위험도	보안약점	원인패턴 건수	탐지건수
CWE-754	중간	비정상적이거나 예외적 인 조건에 대한 부적절한 검사	1	15

2.4 [5.1. Null Pointer 역참조]

보안약점ID	위험도	보안약점	원인패턴 건수	탐지건수
CWE-476	중간	NULL 포인터 역 참조	1	3

2.5 [5.2. 부적절한 자원 해제]

보안약점ID	위험도	보안약점	원인패턴 건수	탐지건수
CWE-404	중간	부적절한 리소스 종료 또는 해제	1	3

3. 프로젝트 점검 항목별 결과 상세 (전체)

3.1 [1.3. 경로 조작 및 자원 삽입]

보안약점	ID	CWE-22	위험도	심각
	제목	1.3. 경로 조작 및 자원 삽입	보안약점 체커	PYTHON_OPEN_FILE_FRO M_EXTERNAL
설명				
소프트웨어는 외부 입력을 사용하여 제한된 상위 디렉토리 아래에있는 파일 또는 디렉토리를 식별하기위한 경로 이름을 구성하지만 소프트웨어는 경로 이름이 다음 위치로 해석되도록 할 수있는 경로 이름 내의 특수 요소를 적절하게 무효화하지 않습니다. 제한된 디렉토리 밖에 있습니다. 많은 파일 작업은 제한된 디렉토리 내에서 수행됩니다. "."및 "/"구분 기호와 같은 특수 요소를 사용하여 공격자는 제한된 위치를 벗어나 시스템의 다른 위치에있는 파일이나 디렉터리에 액세스 할 수 있습니다. 가장 일반적인 특수 요소 중 하나는 "../"시퀀스로, 대부분의 최신 운영 체제에서 현재 위치의 상위 디렉토리로 해석됩니다. 이를 상대 경로 순회라고합니다. 경로 순회는 또한 "/ usr / local / bin"과 같은 절대 경로 이름의 사용을 다루며 이는 예기치 않은 파일에 액세스하는데 유용 할 수도 있습니다. 이를 절대 경로 순회라고합니다. 많은 프로그래밍 언어에서 널 바이트 (0 또는 NUL)를 삽입하면 공격자가 생성 된 파일 이름을 잘라 공격 범위를 넓힐 수 있습니다. 예를 들어 소프트웨어는 모든 경로 이름에 ".txt"를 추가하여 공격자를 텍스트 파일로 제한 할 수 있지만 null 주입은이 제한을 효과적으로 제거 할 수 있습니다.				

해결방안

검증되지 않은 외부 입력값을 통해 파일 및 서버 등 시스템 자원에 대한 접근 혹은 식별을 허용할 경우, 공격자는 입력값 조작을 통해 시스템이 보호하는 자원에 임의로 접근할 수 있는 보안약점입니다.

경로조작 및 자원삽입 약점을 이용하여 공격자는 자원의 수정, 삭제, 시스템 정보누출, 시스템 자원 간 충돌로 인한 서비스 장애 등을 유발시킬 수 있습니다.

즉, 경로 조작 및 자원 삽입을 통해서 공격자가 허용되지 않은 권한을 획득하여, 설정에 관계된 파일을 변경하거나 실행시킬 수 있습니다.

취약한 코드

외부 입력값이 적절한 필터링이나 검증없이 File 함수 인자로 사용되면 임의 파일 삭제, 임의 파일 업로드 등 의도하지 않은 동작을 야기합니다. 이러한 경우를 탐지합니다.

```
import os
from django.shortcuts import render

def bad(request):
    # 외부 입력값으로부터 파일명을 입력 받는다
    request_file = request.POST.get('request_file')

    (filename, file_ext) = os.path.splitext(request_file)
    file_ext = file_ext.lower()

    if file_ext not in ['.txt', '.csv']:
        return render(request, '/error.html', {'error': 'The file could not be opened.'})

    # 입력값을 검증 없이 파일 처리에 사용했다
    with open(request_file) as f:
        data = f.read()

    return render(request, '/success.html', {'data': data})
```

안전한 코드

1. 외부 입력값을 자원(파일, 소켓포트 등)의 식별자로 사용하는 경우, 적절한 검증을 거치도록 하거나 사전에 정의된 인자값만 사용합니다.
2. 외부 입력값이 파일명인 경우에는 경로순회(Directory Traversal) 공격의 위험이 있는 문자(", /, ₩, .. 등)를 제거할 수 있는 필터를 이용하여 제거합니다.
3. 미리 정의된 인자값만 사용합니다.

```
import os
from django.shortcuts import render

def good(request):
    request_file = request.POST.get('request_file')
```

```
(filename, file_ext) = os.path.splitext(request_file)
file_ext = file_ext.lower()

# 외부 입력값으로 받은 파일 이름은 검증하여 사용한다.
if file_ext not in ['.txt', '.csv']:
    return render(request, '/error.html', {'error': 'The file could not be opened.'})

# 파일 명에서 경로 조작 문자열을 필터링 한다.
filename = filename.replace('.', '')
filename = filename.replace('/', '')
filename = filename.replace('\\\\', '')

try:
    with open(filename + file_ext) as f:
        data = f.read()
except IOError:
    return render(request, "/error.html", {"error": "The file either does not exist or cannot be opened."})

return render(request, '/success.html', {'data': data})
```

탐지 번호	376525	신규 / 기존	신규	위험도	심각
보안약점 체크	PYTHON_OPEN_FILE_FROM_EXTERNAL			분석 라인	223
파일	file_translator.py				

```
218:         page_texts = []
219:
220:         # 2) pypdf가 실패했거나 페이지 수를 못 얻은 경우 PyMuPDF로 재시도
221:         if not page_texts and pymupdf is not None:
222:             try:
223:                 pdf_doc = pymupdf.open(stream=pdf_bytes, filetype="pdf")
224:             try:
225:                 for page_num in range(len(pdf_doc)):
226:                     self._check_cancel()
227:                     page_texts.append(pdf_doc[page_num].get_text() or "")
228:             finally:
```

3.2 [4.2. 오류상황 대응 부재]

보안약점	ID	CWE-390	위험도	중간
	제목	4.2. 오류상황 대응 부재	보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT
설명				
소프트웨어는 특정 오류를 감지하지만 오류를 처리하기위한 조치를 취하지 않습니다.				
해결방안				
오류가 발생할 수 있는 부분을 확인하였으나, 이러한 오류에 대하여 예외처리를 하지 않을 경우, 공격자는 오류 상황을 악용하여 개발자가 의도하지 않은 방향으로 프로그램이 동작할 수 있습니다.				
취약한 코드				
try catch나 메소드 반환값에 대해 적절한 예외처리를 하지 않을 경우, 오류 원인에 대한 추적이 불가능합니다. 이러한 경우를 탐지합니다.				
<pre>def bad(): try : num1 = 10 num2 = 0 result = num1 / num2 except ZeroDivisionError as e: pass</pre>				
안전한 코드				
오류가 발생할 수 있는 부분에 대하여 제어문을 사용하여 적절하게 예외 처리(if, switch, try~catch 등)를 합니다.				
<pre>def good(): try : num1 = 10 num2 = 0 result = num1 / num2 except ZeroDivisionError as e: logging.error(e)</pre>				

--	--	--	--	--	--

탐지 번호	376512	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	178
파일	app_ui.py				
<pre>173: # SetProcessDpiAwareness(2) == PER_MONITOR_DPI_AWARE 174: try: 175: ctypes.windll.shcore.SetProcessDpiAwareness(2) 176: except Exception: 177: ctypes.windll.user32.SetProcessDPIAware() 178: except Exception: 179: pass 180: 181: def _apply_responsive_geometry(self) -> None: 182: """현재 화면 크기의 약 85% 로 자동 조정. 작은 화면에서도 잘리지 않게.""" 183: try:</pre>					

탐지 번호	376514	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	210
파일	app_ui.py				
<pre>205: # Tk 스케일링: DPI 기반 보정만 적용. UI 스케일은 명명 폰트로 처리. 206: try: 207: dpi = self.root.winfo_fpixels("1i") # px per inch 208: dpi_scale = max(0.85, min(1.5, dpi / 96.0)) 209: self.root.tk.call("tk", "scaling", dpi_scale) 210: except tk.TclError: 211: pass 212: 213: # ===== 214: # 적응형 폰트 시스템 215: # =====</pre>					

탐지 번호	376515	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	296
파일	app_ui.py				
<pre>291: if event.widget is not self.root: 292: return 293: if self._resize_after_id is not None: 294: try: 295: self.root.after_cancel(self._resize_after_id) 296: except (tk.TclError, ValueError): 297: pass 298: self._resize_after_id = self.root.after(120, self._apply_adaptive_scale) 299: 300: # ===== 301: # UI 구성</pre>					

탐지 번호	376521	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	21
파일	dependencies.py				
<pre>16: PdfReader = None 17: 18: # PyInstaller에서 누락되는 pypdf 서브모듈 강제 import 19: try: 20: import pypdf.filters # noqa: F401 21: except ImportError: 22: pass 23: try: 24: import pypdf._crypt_providers # noqa: F401 25: except ImportError: 26: pass</pre>					

탐지 번호	376522	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	25
파일	dependencies.py				
<pre>20: import pypdf.filters # noqa: F401 21: except ImportError: 22: pass 23: try: 24: import pypdf._crypt_providers # noqa: F401 25: except ImportError: 26: pass</pre>					

```

27:
28: try:
29:     import fitz as pymupdf
30: except ImportError:

```

탐지 번호	376528	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	359
파일	file_translator.py				

```

354:
355:     subprocess.Popen = patched_popen
356:     try:
357:         try:
358:             return pytesseract.image_to_string(img, lang=ocr_lang)
359:         except UnicodeDecodeError:
360:             pass
361:
362:     raw = pytesseract.image_to_string(
363:         img, lang=ocr_lang, output_type=pytesseract.Output.BYTES
364:     )

```

탐지 번호	376531	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	47
파일	main.py				

```

42:     print(f"\n {error_type} (Code: {error_code})")
43:     print("상세 내용은 로그 파일(~/.llm_translator/logs/app.log)을 확인해 주세요.")
44:     _show_error_dialog(error_code, error_type)
45:     try:
46:         input("\nEnter를 눌러 종료하세요...")
47:     except EOFError:
48:         pass
49:     return 1
50:
51:
52: def _show_error_dialog(error_code: str, error_type: str) -> None:

```

탐지 번호	376535	신규 / 기존	신규	위험도	중간
보안약점 चेकर	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	27
파일	runtime_setup.py				
<pre>22: try: 23: cert_path = certifi.where() 24: if cert_path and os.path.exists(cert_path): 25: os.environ["SSL_CERT_FILE"] = cert_path 26: os.environ["REQUESTS_CA_BUNDLE"] = cert_path 27: except (OSError, AttributeError): 28: # certifi 경로 확인 실패는 비치명적 29: pass 30: 31: 32: def _fix_stdio_for_gui() -> None:</pre>					

탐지 번호	376538	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	47
파일	runtime_setup.py				
<pre>42: def _setup_utf8_output_for_windows() -> None: 43: """Windows cp949 인코딩 에러 완화.""" 44: if sys.stdout and hasattr(sys.stdout, "reconfigure"): 45: try: 46: sys.stdout.reconfigure(encoding="utf-8", errors="replace") 47: except Exception: 48: pass 49: if sys.stderr and hasattr(sys.stderr, "reconfigure"): 50: try: 51: sys.stderr.reconfigure(encoding="utf-8", errors="replace") 52: except Exception:</pre>					

탐지 번호	376534	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	52
파일	runtime_setup.py				
<pre>47: except Exception: 48: pass 49: if sys.stderr and hasattr(sys.stderr, "reconfigure"): 50: try: 51: sys.stderr.reconfigure(encoding="utf-8", errors="replace") 52: except Exception: 53: pass</pre>					

탐지 번호	376540	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	102
파일	security.py				
<pre>097: p = Path(path) 098: if p.is_dir(): 099: os.chmod(str(p), stat.S_IRUSR stat.S_IWUSR stat.S_IXUSR) 100: else: 101: os.chmod(str(p), stat.S_IRUSR stat.S_IWUSR) 102: except OSError: 103: # 권한 설정 실패는 비치명적 104: pass 105: 106: 107: def safe_output_path(base_dir: Path, filename: str) -> Path:</pre>					

탐지 번호	376541	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_EMPTY_CATCH_STATEMENT			분석 라인	128
파일	settings_dialog.py				
<pre>123: if event.widget is not self.dialog: 124: return 125: if self._resize_after_id is not None: 126: try: 127: self.dialog.after_cancel(self._resize_after_id) 128: except (tk.TclError, ValueError): 129: pass 130: self._resize_after_id = self.dialog.after(120, self._apply_adaptive) 131: 132: # ----- 133: # UI</pre>					

3.3 [4.3. 부적절한 예외 처리]

보안약점	ID	CWE-754	위험도	중간
	제목	4.3. 부적절한 예외 처리	보안약점 체커	PYTHON_IMPROPER_CATCH_EXCEPTION
설명				
소프트웨어는 소프트웨어의 일상적인 작동 중에 자주 발생하지 않을 것으로 예상되는 비정상적이거나 예외적인 조건을 확인하거나 잘못 확인하지 않습니다. 프로그래머는 메모리 부족 상태, 제한된 권한으로 인한 리소스 액세스 부족, 클라이언트 또는 구성 요소 오작동과 같은 특정 이벤트 또는 조건이 발생하지 않거나 걱정할 필요가 없다고 가정 할 수 있습니다. 그러나 공격자는 의도적으로 이러한 비정상적인 조건을 트리거하여 프로그래머의 가정을 위반하여 불안정성, 잘못된 동작 또는 취약성을 유발할 수 있습니다. 이 항목은 비정상적이거나 예기치 않은 조건을 확인하고 처리하기위한 메커니즘 인 예외 및 예외 처리의 사용에 관한 것이 아닙니다.				

해결방안

Exception 클래스를 이용하여 예외 처리를 광범위하게 처리하는 경우, 탐지예외 처리를 사용하는 경우에 광범위한 예외 처리 대신 구체적인 예외처리를 수행해야 합니다.

취약한 코드

광범위한 예외 클래스인 Exception을 사용하여 예외처리를 하고 있습니다.

```
def bad():
    try:
        list = []
        result = list[0] / x
    except Exception as ex:
        logging.error(ex)
```

안전한 코드

발생 가능한 모든 예외에 대한 구체적인 예외처리를 합니다.

```
def good():
    try:
        list = []
        result = list[0] / x
    except ZeroDivisionError:
        logging.error('Divided by zero')
    except IndexError:
        logging.error('Invalid index')
```

탐지 번호	376511	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	176
파일	app_ui.py				

```
171:     try:
172:         import ctypes
173:         # SetProcessDpiAwareness(2) == PER_MONITOR_DPI_AWARE
174:         try:
175:             ctypes.windll.shcore.SetProcessDpiAwareness(2)
176:         except Exception:
177:             ctypes.windll.user32.SetProcessDPIAware()
178:     except Exception:
```

```

179:         pass
180:
181:     def _apply_responsive_geometry(self) -> None:

```

탐지 번호	376513	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	178
파일	app_ui.py				

```

173:         # SetProcessDpiAwareness(2) == PER_MONITOR_DPI_AWARE
174:         try:
175:             ctypes.windll.shcore.SetProcessDpiAwareness(2)
176:         except Exception:
177:             ctypes.windll.user32.SetProcessDPIAware()
178:         except Exception:
179:             pass
180:
181:     def _apply_responsive_geometry(self) -> None:
182:         """현재 화면 크기의 약 85% 로 자동 조정. 작은 화면에서도 잘리지 않게."""
183:         try:

```

탐지 번호	376516	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	871
파일	app_ui.py				

```

866:         if not DND_AVAILABLE or DND_FILES is None:
867:             return
868:         try:
869:             self.root.drop_target_register(DND_FILES)
870:             self.root.dnd_bind("<<Drop>>", self._on_drop)
871:         except Exception as e:
872:             logger.error(f"DND-INIT: {type(e).__name__}")
873:
874:     def _on_drop(self, event) -> None:
875:         raw = event.data or ""
876:         # tkinterdnd2 파일 경로 파싱 (공백 포함 경로는 중괄호로 감싸짐)

```

탐지 번호	376517	신규 / 기존	신규	위험도	중간
보안약점 चेकर	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	1215
파일	app_ui.py				
<pre>1210: os.startfile(path) # type: ignore[attr-defined] 1211: elif sys.platform == "darwin": 1212: subprocess.run(["open", path], check=False) 1213: else: 1214: subprocess.run(["xdg-open", path], check=False) 1215: except Exception as e: 1216: self.log(f"폴더 열기 실패: {type(e).__name__}", "error") 1217: 1218: def _clear_log(self) -> None: 1219: self.log_text.delete("1.0", tk.END)</pre>					

탐지 번호	376523	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	213
파일	file_translator.py				
<pre>208: self._check_cancel() 209: if cb: 210: cb(page_num + 1, total, f"PDF 텍스트 추출 중... 페이지 {page_num+1}/{total}") 211: try: 212: text = page.extract_text() or "" 213: except Exception: 214: text = "" 215: page_texts.append(text) 216: except Exception: 217: logger.error("PDF-01: pypdf failed") 218: page_texts = []</pre>					

탐지 번호	376524	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	216
파일	file_translator.py				
<pre>211: try: 212: text = page.extract_text() or "" 213: except Exception: 214: text = "" 215: page_texts.append(text) 216: except Exception: 217: logger.error("PDF-01: pypdf failed") 218: page_texts = []</pre>					

```

219:
220:     # 2) pypdf가 실패했거나 페이지 수를 못 얻은 경우 PyMuPDF로 재시도
221:     if not page_texts and pymupdf is not None:

```

탐지 번호	376526	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	230
파일	file_translator.py				

```

225:         for page_num in range(len(pdf_doc)):
226:             self._check_cancel()
227:             page_texts.append(pdf_doc[page_num].get_text() or "")
228:         finally:
229:             pdf_doc.close()
230:     except Exception:
231:         logger.error("PDF-02: PyMuPDF text extraction failed")
232:         page_texts = []
233:
234:     if not page_texts:
235:         raise RuntimeError(

```

탐지 번호	376527	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	333
파일	file_translator.py				

```

328:         img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
329:         pdf_doc.close()
330:         pdf_doc = None
331:         ocr_lang = self.translator.get_ocr_lang()
332:         return self._safe_ocr(img, ocr_lang)
333:     except Exception:
334:         logger.error("ERROR-07: OCR from bytes failed")
335:     finally:
336:         if pdf_doc is not None:
337:             pdf_doc.close()
338:     return ""

```

탐지 번호	376530	신규 / 기존	신규	위험도	중간
보안약점 चेकर	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	484
파일	file_translator.py				

```
479:
480:     img = Image.open(filepath)
481:     ocr_lang = self.translator.get_ocr_lang()
482:     try:
483:         text = self._safe_ocr(img, ocr_lang)
484:     except Exception as ocr_err:
485:         raise RuntimeError(f"OCR 실행 실패 ({type(ocr_err).__name__})")
486:
487:     if cb:
488:         cb(2, 3, "OCR 텍스트 저장 중...")
```

탐지 번호	376529	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	29
파일	logging_config.py				
<pre>24: """로그 메시지의 민감정보를 마스킹.""" 25: 26: def filter(self, record: logging.LogRecord) -> bool: 27: try: 28: msg = record.getMessage() 29: except Exception: 30: return True 31: for pattern, replacement in _SENSITIVE_PATTERNS: 32: msg = pattern.sub(replacement, msg) 33: record.msg = msg 34: record.args = ()</pre>					

탐지 번호	376539	신규 / 기존	신규	위험도	중간
보안약점 체커	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	47
파일	runtime_setup.py				

```
42: def _setup_utf8_output_for_windows() -> None:
43:     """Windows cp949 인코딩 에러 완화."""
44:     if sys.stdout and hasattr(sys.stdout, "reconfigure"):
45:         try:
46:             sys.stdout.reconfigure(encoding="utf-8", errors="replace")
47:         except Exception:
48:             pass
49:     if sys.stderr and hasattr(sys.stderr, "reconfigure"):
```

```

50:     try:
51:         sys.stderr.reconfigure(encoding="utf-8", errors="replace")
52:     except Exception:

```

탐지 번호	376537	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	52
파일	runtime_setup.py				

```

47:     except Exception:
48:         pass
49:     if sys.stderr and hasattr(sys.stderr, "reconfigure"):
50:         try:
51:             sys.stderr.reconfigure(encoding="utf-8", errors="replace")
52:         except Exception:
53:             pass

```

탐지 번호	376542	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	95
파일	time_estimator.py				

```

090:         est = self._est_pdf(filepath, chunk_size)
091:     elif ext in IMAGE_EXTENSIONS:
092:         est = self._est_image(filepath)
093:     else:
094:         return Estimate(error=f"지원 안 함: {ext}")
095:     except Exception as e:
096:         logger.error(f"ESTIM-{ext}: {type(e).__name__}: {e}")
097:         return Estimate(error=type(e).__name__)
098:
099:     avg = self.get_avg_per_chunk(mode)
100:     est.est_seconds = est.ocr_pages * self.OCR_PER_PAGE_S + est.chunks * avg

```

탐지 번호	376543	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	121
파일	time_estimator.py				

```

116:     if size > 1_000_000:
117:         with open(filepath, "rb") as f:
118:             sample = f.read(100_000)
119:         try:
120:             sample_text = sample.decode("utf-8", errors="replace")
121:         except Exception:
122:             sample_text = ""

```



```

123:         ratio = size / max(1, len(sample))
124:         est_chars = int(len(sample_text) * ratio)
125:     else:
126:         est_chars = 0

```

탐지 번호	376544	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_IMPROPER_CATCH_EXCEPTION			분석 라인	213
파일	time_estimator.py				

```

208:     sample_chars = 0
209:     sample_scan_pages = 0
210:     for i in range(sample_n):
211:         try:
212:             t = (reader.pages[i].extract_text() or "").strip()
213:         except Exception:
214:             t = ""
215:         if len(t) < 10:
216:             sample_scan_pages += 1
217:         else:
218:             sample_chars += len(t)

```

3.4 [5.1. Null Pointer 역참조]

보안약점	ID	CWE-476	위험도	중간
	제목	5.1. Null Pointer 역참조	보안약점 체크	PYTHON_OBJECT_REFERENCE_NULL_DEREFERENCE
설명				
NULL 포인터 역 참조는 응용 프로그램이 유효 할 것으로 예상되는 포인터를 역 참조 할 때 발생하지만 일반적으로 충돌 또는 종료를 유발하는 NULL입니다. NULL 포인터 역 참조 문제는 경쟁 조건 및 간단한 프로그래밍 누락을 포함한 여러 결함을 통해 발생할 수 있습니다.				
해결방안				
널 포인터(Null Pointer) 역참조는 '일반적으로 그 객체가 널(Null)이 될 수 없다'라고 하는 가정을 위반했을 때 발생한다. 공격자가 의도적으로 널 포인터 역참조를 발생시키는 경우, 공격자는 그 결과로 발생하는 예외 상황을 이용해 추후 공격 계획에 활용할 수 있다. 파이썬에서는 Null pointer dereference가 발생하지 않는다. 파이썬에서는 Null 객체가 사용되지 않으며 대신 None 키워드를 사용해 null 개체와 변수를 정의 한다. None은 다른 언어의 null과 동일한 기능을 수행 하지 않으며 None이 0 또는 다른 값을 정의 하진 않는다. None을 반환하는 함수를 사용하면 None과 다른 값(예: 0이나 빈 문자열)이 조건문에서 False로 평가될 수 있기 때문에 실수하기 쉽다. None이 될 수 있는 데이터를 참조하기 전에 해당 데이터의 값이 None 인지 검사하여 시스템 오류를 줄일 수 있다.				
취약한 코드				
파이썬에서는 포인터를 사용하지는 않지만 데이터에 대한 적절한 검사를 수행하지 않을 경우 Null pointer와 유사한 None 값 참조 오류를 범할 수 있다. <pre> from flask import Flask, request app = Flask(__name__) @app.route('/bad', methods=['GET']) def bad(): # 외부 입력을 처리하는 부분 data = request.args.get('data') # 데이터가 None인지 체크하지 않고 메서드를 호출함 result = data.strip() return f"Processed data: {result}" </pre>				
안전한 코드				

참조하고자 하는 자원을 호출 시에는 반드시 개체가 None이 아닌지 검증해야 한다.

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/good', methods=['GET'])
def good():
    # 외부 입력을 처리하는 부분
    data = request.args.get('data')

    # None인 경우를 처리
    if data is None:
        return "No data provided", 400

    result = data.strip()
    return f"Processed data: {result}"
```

탐지 번호	376518	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_OBJECT_REFERENCE_NULL_DEREFERENCE			분석 라인	1282
파일	app_ui.py				
1277:	results["outputs"].append(out)				
1278:	self.file_status[fp] = STATUS_DONE				
1279:	# 실측 시간으로 모드별 평균 갱신 (OCR 시간은 별도 가중치이므로 제외)				
1280:	duration = time.time() - file_start				
1281:	est = self.file_estimates.get(fp)				
1282:	if est and est.chunks > 0 and duration > 0:				
1283:	api_secs = max(0.0,				
1284:	duration - est.ocr_pages * self.time_estimator.OCR_PER_PAGE_S)				
1285:	self.time_estimator.update_avg(				
1286:	self.mode_var.get(), api_secs, est.chunks				
1287:	)				

탐지 번호	376519	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_OBJECT_REFERENCE_NULL_DEREFERENCE			분석 라인	1284
파일	app_ui.py				
1279:	# 실측 시간으로 모드별 평균 갱신 (OCR 시간은 별도 가중치이므로 제외)				
1280:	duration = time.time() - file_start				
1281:	est = self.file_estimates.get(fp)				
1282:	if est and est.chunks > 0 and duration > 0:				
1283:	api_secs = max(0.0,				

```
1284:         duration - est.ocr_pages * self.time_estimator.OCR_PER_PAGE_S)
1285:         self.time_estimator.update_avg(
1286:             self.mode_var.get(), api_secs, est.chunks
1287:         )
1288:         self._log(f" 완료 → {Path(out).name} ({format_secs(duration)})",
1289:             "success")
```

탐지 번호	376520	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_OBJECT_REFERENCE_NULL_DEREFERENCE			분석 라인	1286
파일	app_ui.py				
<pre>1281: est = self.file_estimates.get(fp) 1282: if est and est.chunks > 0 and duration > 0: 1283: api_secs = max(0.0, 1284: duration - est.ocr_pages * self.time_estimator.OCR_PER_PAGE_S) 1285: self.time_estimator.update_avg(1286: self.mode_var.get(), api_secs, est.chunks 1287:) 1288: self._log(f" 완료 → {Path(out).name} ({format_secs(duration)})", 1289: "success") 1290: except TranslationCancelled: 1291: results["cancelled"] += 1</pre>					

3.5 [5.2. 부적절한 자원 해제]

보안약점	ID	CWE-404	위험도	중간
	제목	5.2. 부적절한 자원 해제	보안약점 체커	PYTHON_RESOURCE_RELEASE_NOT_FINALLY
설명				
프로그램은 재사용이 가능해지기 전에 자원을 해제하지 않거나 잘못 해제합니다. 리소스가 생성되거나 할당되면 개발자는 리소스를 적절하게 해제하고 설정된 기간 또는 해지와 같은 만료 또는 무효화의 모든 잠재적 경로를 고려해야 합니다.				
해결방안				
프로그램의 자원, 예를 들면 열려 있는 파일 식별자(Open File Descriptor), 힙 메모리(Heap Memory), 소켓(Socket) 등은 유한한 자원이다. 이러한 자원을 할당 받아 사용을 마치고 더 이상 사용하지 않는 경우에는 적절히 반환해야 하는데, 프로그램 오류 또는 예외로 사용이 끝난 자원을 반환하지 못하는 경우에 문제가 발생 할 수 있다. 자원을 획득하여 사용한 다음에는 반드시 자원을 해제 후 반환한다.				
취약한 코드				
다음은 try 구문 내의 코드 실행 중 오류가 발생할 경우, close() 메소드가 실행되지 않아 사용한 자원이 반환되지 않는 경우를 보여 준다. <pre>def bad(file_path): try: file = open(file_path, 'r') data = file.read() # 파일을 닫는 부분이 명시적으로 호출되지 않음 except FileNotFoundError as e: logging.error(e) return data</pre>				
안전한 코드				
예외 상황이 발생하여 함수가 종료될 때, 예외의 발생 여부와 상관없이 항상 실행되는 finally 블록에서 할당 받은 모든 자원을 반환해야 한다. 다른 방법은 with 문을 사용해 파일을 처리하는 방법으로, with 문의 블록이 끝날 때 자동으로 파일 자원을 반환하는 예시다. 이렇게 작성하면 with문 내의 코드에 예외가 발생하더라도 항상 파일 닫기가 보장된다.				

```
def good1(file_path):
    with open(file_path, 'r') as file:
        data = file.read()
    return data

def good2(file_path):
    try:
        file = open(file_path, 'r')
        data = file.read()
    except FileNotFoundError as e:
        logging.error(e)
    finally:
        file.close()
    return data
```

탐지 번호	376533	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_RESOURCE_RELEASE_NOT_FINALLY			분석 라인	35
파일	runtime_setup.py				

```
30:
31:
32: def _fix_stdio_for_gui() -> None:
33:     """GUI 배포 시 stdin/stdout/stderr None 방지."""
34:     if sys.stdout is None:
35:         sys.stdout = open(os.devnull, "w")
36:     if sys.stderr is None:
37:         sys.stderr = open(os.devnull, "w")
38:     if sys.stdin is None:
39:         sys.stdin = open(os.devnull, "r")
```

탐지 번호	376532	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_RESOURCE_RELEASE_NOT_FINALLY			분석 라인	37
파일	runtime_setup.py				

```
32: def _fix_stdio_for_gui() -> None:
33:     """GUI 배포 시 stdin/stdout/stderr None 방지."""
34:     if sys.stdout is None:
35:         sys.stdout = open(os.devnull, "w")
36:     if sys.stderr is None:
37:         sys.stderr = open(os.devnull, "w")
38:     if sys.stdin is None:
39:         sys.stdin = open(os.devnull, "r")
40:
```

```

41:
42: def _setup_utf8_output_for_windows() -> None:

```

탐지 번호	376536	신규 / 기존	신규	위험도	중간
보안약점 체크	PYTHON_RESOURCE_RELEASE_NOT_FINALLY			분석 라인	39
파일	runtime_setup.py				

```

34:     if sys.stdout is None:
35:         sys.stdout = open(os.devnull, "w")
36:     if sys.stderr is None:
37:         sys.stderr = open(os.devnull, "w")
38:     if sys.stdin is None:
39:         sys.stdin = open(os.devnull, "r")
40:
41:
42: def _setup_utf8_output_for_windows() -> None:
43:     """Windows cp949 인코딩 에러 완화."""
44:     if sys.stdout and hasattr(sys.stdout, "reconfigure"):

```